

CSC 212: Data Structures and Abstractions

01: Introduction

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



CSC 212

Learn to model and solve complex problems with computers.

The fundamentals that let you design, evaluate, and VERIFY algorithms

Course description

- Introduction to **fundamental data structures** and their algorithms
 - ✓ arrays, lists, stacks, queues, trees, hash tables, graphs (most popular topics for job interview questions)
 - ✓ survey of classic algorithms for sorting and searching
- Basic principles of analysis of algorithms
 - ✓ improve your foundation of CS theory
- Writing code that runs efficiently
 - ✓ choosing good algorithms and data structures

Presumes a strong foundation in C++ programming, including:

- Pointers
- Classes and objects
- Recursion

3

Why this course in the AI era?

- What does AI do well?
 - ✓ syntax, boilerplate, API recall, translating between languages
 - ✓ explaining a concept five different ways until one lands
 - ✓ decoding a 400-line C++ template error
- Where AI might be unreliable
 - ✓ choosing the right structure for a problem, given real constraints
 - ✓ asymptotic claims
 - ✓ C++ memory semantics: ownership, lifetimes, dangling pointers, copy costs
 - ✓ verification
- The job changed shape, not difficulty
 - ✓ adapting from coding => specifying, reviewing, and being accountable for it

Every system you will build, including AI systems, is made of data structures. Embeddings live in arrays. Vector search is a nearest-neighbor structure. A tokenizer is a trie. A KV-cache is a hash table with an eviction policy.

4

Course information

- Course Website
 - ✓ Syllabus, Schedule, Resources
 - ✓ <https://homepage.cs.uri.edu/~malvarez/teaching/csc-212/>

5

What is expected from you?

- Attend lectures/labs
 - ✓ students are expected to attend all lectures and labs
 - ✓ regular attendance is linked to higher grades and better comprehension of course material
- Participate and think critically
 - ✓ ask questions (lectures, labs, office hours, Ed, ...)
- Start working on assignments early
 - ✓ refrain from simply copying and pasting answers generated by LLMs
- Laptops and cellphones are **NOT permitted** unless being used for taking notes

6

Q1-1

- According to the syllabus, what happens to a homework assignment submitted one day late?
 - A. A 10% grade penalty is applied
 - B. It is not accepted
 - C. It is accepted with instructor approval only
 - D. It is accepted within a 24-hour grace period
 - E. None of the above

7

Answer

- B) It is not accepted
 - ✓ late submissions are not accepted, instead, you get 6-9 days of lead time on every assignment, so start early and use office hours if you're stuck

8

Q1-2

▸ To pass this class, which of these must be true, regardless of your final grade?

- A. You attended at least 80% of lectures
- B. Your assignment average is ≥ 50
- C. Your average across all exams is ≥ 50
- D. You scored ≥ 50 on the final exam alone
- E. Not enough information

9

Answer

▸ B) Your average across all exams is ≥ 50

✓ exams are where the course verifies you actually learned the material

10

Need a refresher on C++ programming?

▸ Pick a textbook (learn syntax)

▸ Solve Challenges



Kattis



CODEFORCES

Sponsored by TON



codewars



LeetCode



topcoder™

11

Q1-3

▸ What is wrong with this code?

```
void grow(int *arr, int n) {  
    arr = new int[2 * n];  
}  
  
int main() {  
    int *data = new int[10];  
    grow(data, 10);  
    data[15] = 42; // <-- what happens here?  
}
```

- A. Nothing, `grow` correctly reallocates the array
- B. `grow` receives a copy of `data`, so `data` still points to the old array, then `data[15]` writes out of bounds
- C. `new int[2*n]` is invalid C++, the size must be a compile-time constant
- D. The code leaks memory but is otherwise correct
- E. None of the above

12

Answer

- B) **grow** receives a copy of **data**, so **data** still points to the old array, then **data[15]** writes out of bounds

- ✓ assigning to **arr** rebinds the local copy and has no effect on **data** in main
- ✓ consequences, in order of severity:
 - **data** still points to the original 10-element block, so **data[15]** writes past the end, undefined behavior, and it may not crash
 - the freshly allocated block is leaked, nothing points to it
 - neither block is ever **delete[]**-ed

13

Q1-4

- What is the behavior of this program?

```
std::vector<int> v = {1, 2, 3, 4, 5};
int& r = v[0];
v.push_back(6);
std::cout << r; // <-- what does this print?
```

- A. Prints 1, **r** refers to the first element, which is unchanged
- B. Prints 6
- C. Compile error, you cannot bind a reference to **v[0]**
- D. Undefined behavior, **push_back** may reallocate, leaving **r** dangling
- E. None of the above

14

Answer

- D) Undefined behavior, **push_back** may reallocate, leaving **r** dangling

- ✓ **push_back** may exceed the vector's capacity, in which case it allocates a new buffer, copies the elements over, and frees the old one
- ✓ any reference, pointer, or iterator into the old buffer, including **r**, is left dangling
- ✓ it might print 1. It might print garbage. It might work every time you test it and fail in production
- ✓ A) is so tempting; the value at index 0 really is unchanged. But **r** does not refer to 'index 0', it refers to a memory address, and that address is no longer yours

15

Warming up

- Adjacent elements sum
 - ✓ find the **maximum sum** of any pair of adjacent elements in an array of integers

14

11	3	5	3	2	-5	6	7	-9	-2	13	2
----	---	---	---	---	----	---	---	----	----	----	---

first pair

for this example, what is the maximum sum of two adjacent elements?

16

Q1-5

› Complete the following program:

```
#include <vector>
#include <iostream>

int max_sum(std::vector<int>& nums) {
    // TODO
    return -1;
}

int main() {
    std::vector<int> nums = {1, -2, 3, 4, -1, 2, 1, -5, 4};
    int result = max_sum(nums);
    std::cout << "Maximum sum is: " << result << std::endl;
    return 0;
}
```

17

Q1-6

› A student implements `max_sum` by initializing `best` with `-1`, then looping over adjacent pairs, keeping the largest sum. On which input does this return a wrong answer?

- A. It never fails, `-1` is a safe starting value
- B. `{-5, -3, -8, -3}`, every pair sums to a negative number
- C. `{0, 0, 0, 0}`, every pair sums to zero
- D. `{1, 2, 3, 4}`, the pairs are positive
- E. None of the above

18

Answer

› B) `{-5, -3, -8, -3}`, every pair sums to a negative number

- ✓ initializing `best = -1` assumes the answer is at least `-1`
- ✓ the bug is invisible on most inputs, it fires only when every adjacent pair sums below `-1`, so it survives casual testing indefinitely
- ✓ **the fix**: always initialize from the data, not from a guess

```
int best = nums[0] + nums[1]; // must also handle n < 2 first
for (size_t i = 1 ; (i + 1) < nums.size() ; ++i)
    best = std::max(best, nums[i] + nums[i+1]);
```

19